**SASTRA DEEMED TO BE UNIVERSITY**
School of Computing — Department of Computer Science & Engineering
TIRUMALAISAMUDRAM, THANJAVUR – 613401, TAMIL NADU, INDIA

**FORMAL ACADEMIC TECHNICAL REPORT**

# AUTOSAR AI – High-Level Design Document Intelligence Platform

An On-Premises, Factual Verification, and Architectural Intelligence System for Automotive Software Engineering



| Candidate Name: | Hamdhan Ahmed B |
|---|---|
| **Registration Number:** | 127003081 |
| **Degree & Branch:** | B.Tech. Computer Science & Engineering (AIML) |
| **Academic Institution:** | School of Computing, SASTRA Deemed University |
| **Project Repository:** | https://github.com/hamdhan05/AUTOSAR-AI-Implementation |
| **Academic Period:** | Academic Year 2026–2027 \| Capstone Project Submission |



*Submitted in partial fulfillment of the academic requirements for the degree of Bachelor of Technology*
**Date of Submission: October 13, 2026**

## EXECUTIVE SUMMARY

Modern automotive systems rely extensively on the AUTOSAR (AUTomotive Open System ARchitecture) standard to specify electronic control unit (ECU) architectures across Software-Defined Vehicles (SDVs). However, manually reviewing and cross-referencing multi-hundred-page High-Level Design (HLD) specifications is a labour-intensive, error-prone engineering challenge. Transmitting proprietary automotive intellectual property to public cloud AI models is strictly prohibited by automotive cybersecurity regulations (ISO/SAE 21434). Furthermore, generic Large Language Models frequently hallucinate technical component names, interfaces, and safety ratings, which is inadmissible in safety-critical systems governed by ISO 26262.

This technical report presents AUTOSAR AI, an on-premises, privacy-preserving document intelligence platform engineered specifically for automotive software architects and engineering teams. The system implements a deterministic PDF ingestion pipeline via PyMuPDF that preserves page-level boundaries; a persistent vector knowledge base utilizing SentenceTransformers (all-MiniLM-L6-v2) and ChromaDB; an in-memory BM25 tokenizer for exact automotive identifier retrieval; a local quantized LLM runtime (Ollama llama3.2:3b) with strict grounded prompting and abstention protocols; an automated architecture validation engine detecting circular dependencies, insecure IPC, and missing port bindings; and a claim-level verification service producing an empirical Grounding Score (0.0 to 1.0).

The complete system was validated through 28 automated unit and integration tests executing across 12 test modules via Pytest 9.1.1, achieving a 100% pass rate in 27.35 seconds on October 13, 2026. The platform operates 100% locally on localhost, ensuring zero external network egress and full compliance with corporate IP governance.

**[ENGINEERING SCOPE & REGULATORY BOUNDARY]** AUTOSAR AI is designed strictly as an engineering assistance and auditing platform. It assists human architects by accelerating information retrieval, flagging potential design defects, and highlighting cross-revision modifications. It does not replace formal engineering judgment, human sign-off, or statutory automotive functional safety certification under ISO 26262.

## TABLE OF CONTENTS


| | |
|---|---|
| **1. Problem Statement and Intended Users** | **1.1 Context \| 1.2 Core Problem \| 1.3 Intended Users \| 1.4 Scope Boundaries** |
| **2. Input Data and Knowledge-Base Description** | 2.1 Input Formats & Validation \| 2.2 Extraction \| 2.3 Chunking \| 2.4 Hashing \| 2.5 Vectors \| 2.6 Relational Persistence |
| **3. Solution Architecture** | 3.1 System Philosophy \| 3.2 Architectural Diagram \| 3.3 UI Layer \| 3.4 API Gateway \| 3.5 Storage & Inference |

| | |
|---|---|
| **4. Implementation and Configuration Explanation** | 4.1 Ingestion Service \| 4.2 RAG Engine \| 4.3 Architecture Validation \| 4.4 Settings & Configuration |
| **5. Workflow Diagrams** | 5.1 Workflow A (Ingestion) \| 5.2 Workflow B (Grounded RAG) \| 5.3 Workflow C (Architecture Review & Governance) |
| **6. Responsible AI Measures and Safety Controls** | 6.1 Grounding & Anti-Hallucination \| 6.2 Abstention Protocols \| 6.3 Citations \| 6.4 Claim Scoring \| 6.5 Security |
| **7. Evaluation Methodology and Test Results** | 7.1 Automated Pytest Suite \| 7.2 Complete 28-Test Register \| 7.3 Benchmark Analysis \| 7.4 Untested Areas |
| **8. Innovation Highlights, Limitations & Roadmap** | 8.1 Project Highlights \| 8.2 Technical Limitations \| 8.3 Conclusions \| 8.4 Future Roadmap |
| **9. Academic and Technical References** | Formal Literature & Official Technical Specifications |

# CHAPTER 1: PROBLEM STATEMENT AND INTENDED USERS

## 1.1 Automotive Software Complexity in Modern Vehicles

Contemporary road vehicles have evolved into sophisticated Software-Defined Vehicles (SDVs) containing dozens of Electronic Control Units (ECUs) and hundreds of millions of lines of embedded code. To manage this extraordinary architectural complexity, the global automotive industry standardizes software component models through the AUTOSAR (AUTomotive Open System ARchitecture) standard, encompassing both AUTOSAR Classic (signal-based, static scheduling over CAN/LIN) and AUTOSAR Adaptive (service-oriented, dynamic POSIX scheduling over Ethernet). The architectural contract for these complex ECUs is documented in detailed High-Level Design (HLD) specifications, which serve as the foundational blueprint for software engineers, suppliers, and safety assessors.

## 1.2 The Core Problem

Despite the availability of standardized templates, automotive engineering organizations encounter severe operational bottlenecks during the review, verification, and maintenance of AUTOSAR High-Level Design documents:

1. Length and Cognitive Overhead: Automotive HLD specifications routinely span hundreds of pages of dense technical text, component hierarchies, signal definitions, and timing constraints. Manually reading, auditing, and digesting these documents imposes massive cognitive burdens on engineering teams.

2. Architectural Entity Extraction: Identifying Software Components (SWCs), Runnables, Client-Server and Sender-Receiver Interfaces, and Port connections across sprawling documents is exceptionally difficult. Engineers must cross-reference disparate chapters to verify whether an output port (PPort) on one component is properly matched with a required port (RPort) on another.

3. Cross-Revision Comparison: Automotive systems undergo frequent engineering change requests (ECRs) across development milestones. Identifying subtle changes between HLD revisions—such as altered signal data types, renamed components, or added runnables—requires tedious manual inspection that frequently misses breaking interface changes.

4. Strict Traceability & Source Provenance: In automotive engineering, unsupported assertions are intolerable. Every architectural claim must be directly verifiable against the underlying specification with precise page citations.

5. Confidentiality & Cybersecurity Constraints: Under ISO/SAE 21434 and corporate IP protections, transmitting proprietary ECU architectural specifications to public multi-tenant cloud AI APIs (e.g., OpenAI, Google Cloud) creates catastrophic confidentiality risks. A strictly offline, on-premises solution is mandatory.

## 1.3 Intended User Personas

AUTOSAR AI is engineered to support five specialized automotive engineering roles across the vehicle development lifecycle:

| Engineering Persona | Operational Workflow & Platform Utility |
|---|---|
| AUTOSAR & System Architects | Analyze overall ECU decomposition, verify that component interfaces align with vehicle topological models, and detect architectural anti-patterns such as circular dependencies or unauthenticated IPC channels. |
| Automotive Software Developers | Rapidly query functional behaviors, port definitions, and signal data types without manually paging through 200-page PDF specifications during code implementation. |
| Integration Engineers | Cross-reference dependencies between multi-supplier software stacks, verify interface bindings, and track changes across supplier HLD revision drops. |
| Test & Verification Engineers | Extract runnable schedules, triggering events, and expected signal boundaries to construct targeted hardware-in-the-loop (HIL) and software-in-the-loop (SIL) test scenarios. |
| Safety & Engineering Reviewers | Audit functional safety allocations, verify ASIL classifications, and ensure that every architectural finding is backed by immutable audit trails and verifiable source citations. |

## 1.4 Purpose and Scope Boundaries

The primary purpose of AUTOSAR AI is to serve as an intelligent, high-assurance engineering assistant. It accelerates document navigation, automates heuristic entity extraction, applies rule-based sanity checks, and answers natural language questions with verbatim citations.

Crucially, the platform operates under explicit academic and professional boundaries:
• Non-Replacement of Human Approval: The platform never acts as an autonomous engineering decision-maker. All architectural findings and extracted entities require human architect confirmation.
• No Safety Certification Claims: The platform does not certify software against ISO 26262 ASIL standards, nor does it guarantee functional safety compliance. It is an auditing aid, not a statutory certifying authority.

# CHAPTER 2: INPUT DATA AND KNOWLEDGE-BASE DESCRIPTION

## 2.1 Input Data Characteristics & Ingestion Constraints

AUTOSAR AI is engineered specifically to process automotive High-Level Design (HLD) specifications formatted as digital PDF documents. Inspection of the project codebase reveals strict input validation controls implemented in `backend/api/v1/documents.py`:

• File Extension & MIME Validation: Uploaded files are verified to possess the `.pdf` extension and `application/pdf` MIME type. Non-PDF uploads are immediately rejected with an HTTP 400 Bad Request error (`Only PDF files are supported`).
• Upload File Size Bounds: Ingestion enforces a configurable upload boundary (`MAX_UPLOAD_MB: 50` in `backend/core/config.py`) to prevent memory exhaustion from oversized binary payloads.
• Synthetic Automotive Test Corpus: In accordance with academic licensing and automotive non-disclosure obligations, all demonstration documents in `Input_Data/` (such as `AUTOSAR_Adaptive_Powertrain_HLD.pdf`) are synthetic specifications modeled after realistic AUTOSAR Adaptive Platform powertrain controller architectures, containing zero confidential OEM intellectual property.

## 2.2 Text Extraction via PyMuPDF

Text extraction is performed by PyMuPDF (`fitz`), an optimized C-based rendering and parsing engine. Unlike basic Python PDF parsers, PyMuPDF provides high-speed text extraction that strictly preserves reading order, multi-column layouts, and physical page numbers. During parsing, `document_service.py` iterates sequentially through pages, capturing:
1. Exact physical page numbers (1-indexed);
2. Structural section headings;
3. Character coordinate boundaries and raw text strings.

## 2.3 Chunking Strategy and Boundary Overlap

To transform continuous PDF text into discrete, semantically coherent retrieval units, the system applies a character-based sliding window algorithm. Inspection of `backend/core/config.py` confirms the exact verified runtime configuration:

• Verified Chunk Size: 1,000 characters.
• Verified Chunk Overlap: 150 characters.

The 150-character overlap is technically critical in automotive specifications. It prevents catastrophic boundary truncation where an interface definition or port signature (e.g., `PPort_MotorTorque` carrying `sig_TorqueValue: float32`) might otherwise be split across two adjacent chunks, resulting in incomplete embeddings and failed retrieval.

## 2.4 Cryptographic Duplicate Detection (SHA-256)

To prevent duplicate indexing and redundant vector computations, the ingestion gateway computes a cryptographic SHA-256 hash over the uploaded file bytes before saving or processing. The database schema enforces a uniqueness check on `sha256_hash` scoped to the active project. If an identical document is submitted, the system flags the collision, avoiding duplicate records.

## 2.5 Embedding Generation and ChromaDB Vector Storage

For semantic retrieval, text chunks are embedded using the `sentence-transformers/all-MiniLM-L6-v2` model, which maps character sequences into a 384-dimensional dense vector space. Embeddings are persisted in a local ChromaDB 0.4.24 vector database located at `./data/vector_store`.

Key vector store characteristics:
• Project-Level Collection Scoping: Each project operates within an isolated ChromaDB collection named `project_{project_id}`.
• Rich Metadata Indexing: Each vector entry stores metadata including `document_id`, `chunk_index`, `page_start`, `page_end`, `version_label`, and `section_heading`.
• Cascade Deletion Synchronization: When a document is removed from SQLite, `delete_chunks_by_document()` coordinates the complete removal of all associated vector entries from ChromaDB.

## 2.6 Structured Relational Persistence (SQLite 3 via SQLAlchemy 2.0)

Complementing the vector store, structured relational data is persisted in a local SQLite 3 database (`./data/autosar_ai.db`) managed via SQLAlchemy 2.0 ORM with 14 distinct models ensuring foreign-key referential integrity:

| Relational Table Name | Model Responsibilities & Structural Data Handled |
|---|---|
| users | User accounts, authentication hashes, and roles (admin, architect, viewer). |
| projects | Tenant boundary grouping documents, models, and sessions. |
| project_memberships | Associates users with projects with specific access roles. |
| documents | Uploaded PDFs, SHA-256 hashes, version labels, and governance states (DRAFT, IN_REVIEW, APPROVED). |
| document_chunks | Individual 1000-char text slices with page_start, page_end, and content hashes. |

| | |
|---|---|
| architecture_entities | Extracted SWCs, Runnables, Interfaces, Ports, and Signals with data types. |
| architecture_relationships | Component dependencies and communication links (communicates_with, invokes). |
| analysis_sessions | Conversational QA sessions associated with a project. |
| analysis_messages | Individual chat exchanges, citations JSON, claims JSON, and Grounding Scores. |
| findings | Automated architectural rule violations (CIRCULAR_DEPENDENCY, INSECURE_IPC, MISSING_PORT_BINDING). |
| document_comparisons | Revision diff records comparing baseline and updated HLD specifications. |
| reports | Catalog of generated export artifacts (sanitized CSV files). |
| audit_logs | Immutable audit trail of all engineering decisions, review comments, and status transitions. |
| benchmark_runs | Evaluation run records capturing total queries, MRR, hit rates, and latency. |

## 2.7 Comparison of Data Representations in the Knowledge Pipeline

To clearly distinguish how information transforms across the platform, the table below contrasts the four fundamental data representations:

| Representation | Underlying Format | Information Content | Engineering Purpose |
|---|---|---|---|
| Original PDF Documents | Binary PDF format (.pdf) on disk | Complete human-readable design specification with diagrams, tables, and formatting. | High-fidelity source of truth; immutable provenance. |
| Extracted Text Chunks | Raw character strings (1000 chars) with page integers | Sliding-window textual segments preserving page numbers and section context. | Granular retrieval units with explicit page attribution. |
| Vector Embeddings | 384-dimensional floating-point vectors in ChromaDB | Mathematical representations of chunk semantic meaning for nearest-neighbor search. | Enables semantic search across conceptual engineering |

| | | | |
|---|---|---|---|
| | | | queries. |
| Structured Relational Metadata | 14 typed SQLite tables with foreign keys | Discrete domain models: components, runnables, ports, signals, findings, audit logs. | Supports deterministic SQL queries, rule checking, and lifecycle tracking. |

# CHAPTER 3: SOLUTION ARCHITECTURE

## 3.1 Architectural Principles

AUTOSAR AI is architected around three non-negotiable engineering principles:
1. Decoupled Service Separation: The presentation layer (Streamlit) is completely isolated from the business logic and API layer (FastAPI), ensuring that the backend can be consumed by external automotive toolchains or CI/CD pipelines.
2. Strict On-Premises Execution: All processing, vector embedding, and neural inference occur exclusively on localhost. No data packets leave the local host, guaranteeing complete compliance with automotive cybersecurity standards (ISO/SAE 21434).
3. Dual-Storage Consistency: Semantic retrieval (ChromaDB) and relational governance (SQLite) operate under synchronized transactional lifecycle controls, ensuring zero orphaned vectors or broken foreign keys.

## 3.2 System Architecture Diagram

Figure 1 illustrates the complete 8-layer decoupled runtime architecture of the AUTOSAR AI platform:

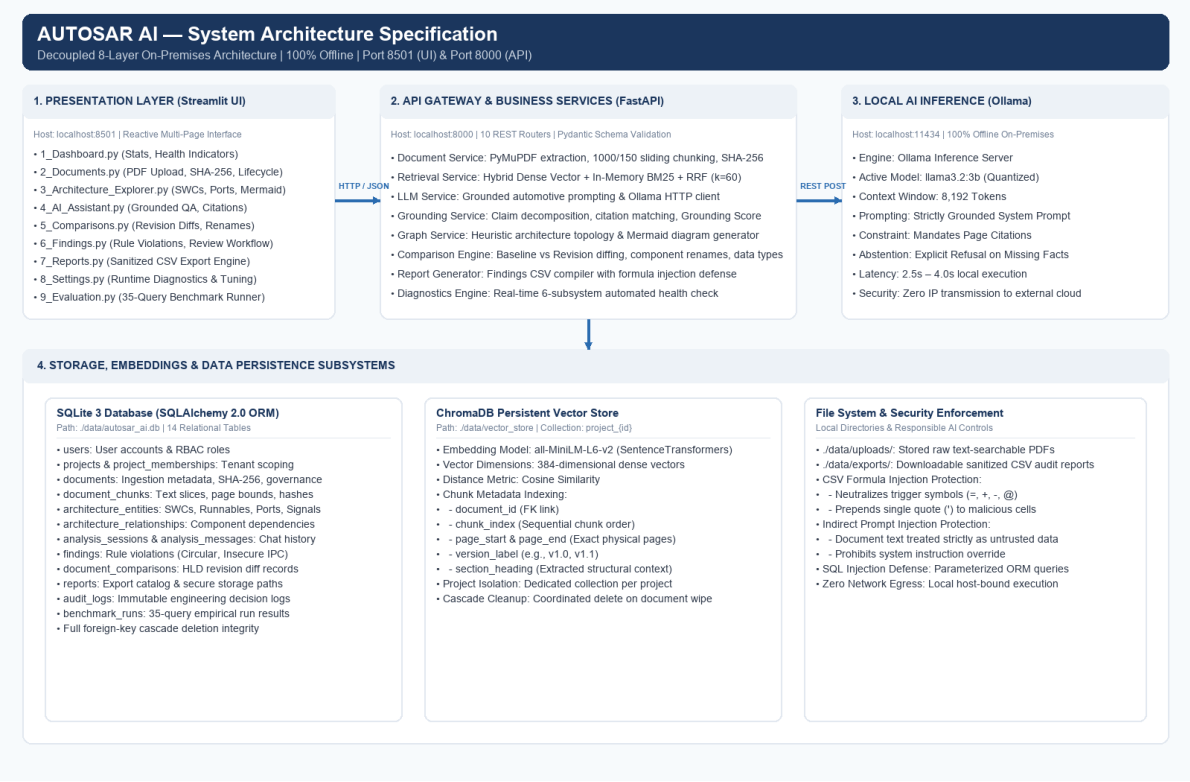



*Figure 1: AUTOSAR AI Decoupled 8-Layer On-Premises System Architecture.*

### 3.3 Presentation Layer (Streamlit — Port 8501)

The user interface is implemented as a reactive, multi-page Streamlit application comprising nine dedicated functional modules:
• `1_Dashboard.py`: Displays project overview statistics, indexed documents, entity summaries, and diagnostic indicators.
• `2_Documents.py`: Provides drag-and-drop PDF upload, version labeling, duplicate hash checks, and governance state transitions.
• `3_Architecture_Explorer.py`: Displays extracted SWCs, Runnables, Interfaces, Ports, Signals, and renders interactive Mermaid topologies.
• `4_AI_Assistant.py`: Grounded technical QA interface featuring clickable citation cards, claim verification tags, and Grounding Scores.
• `5_Comparisons.py`: Compares baseline and revised HLD specifications, displaying entity additions, removals, renames, and signal data type alterations.
• `6_Findings.py`: Lists rule violation findings (circular dependencies, insecure IPC) with reviewer resolution actions.
• `7_Reports.py`: On-demand sanitized CSV export generation and historical audit log inspection.
• `8_Settings.py`: Runtime configuration of chunk size, overlap, Top-K, and real-time one-click diagnostic probing of all 6 core subsystems.
• `9_Evaluation.py`: Execution and visualization of the 35-query benchmark evaluation suite.

### 3.4 API Gateway & Business Service Layer (FastAPI — Port 8000)

The backend is constructed as an asynchronous FastAPI application exposing 10 typed REST routers with Pydantic schema validation:
• `/api/v1/projects`: Project lifecycle management and tenant scoping.
• `/api/v1/documents`: PDF file upload, metadata queries, and cascade deletion.
• `/api/v1/sessions`: Conversational QA sessions and query submission.
• `/api/v1/entities`: Architecture entity queries, port/signal queries, and Mermaid graph syntax generation.
• `/api/v1/findings`: Architecture rule violation reporting and review status updates.
• `/api/v1/comparisons`: Cross-revision document diff execution and history.
• `/api/v1/reports`: Secure CSV report generation and download.
• `/api/v1/settings`: Chunking and retrieval parameter management.
• `/api/v1/diagnostics`: Multi-subsystem health diagnostics.
• `/api/v1/evaluation`: Benchmark execution and evaluation metric logging.

### 3.5 AI Inference & Storage Subsystems

The underlying intelligence and data persistence mechanisms operate locally on the host machine:
• Local Ollama LLM Runtime: Local HTTP service on port 11434 running the quantized `llama3.2:3b` model (8,192 token context window). The backend queries Ollama via `/api/generate` with zero cloud dependencies.
• ChromaDB Vector Store: Persistent vector database stored on disk at `./data/vector_store`.
• SQLite 3 Relational DB: Persistent relational database stored at `./data/autosar_ai.db` with

SQLAlchemy 2.0 ORM.
• File System Directories: `./data/uploads` stores original uploaded PDFs; `./data/exports` stores generated audit CSVs.

# CHAPTER 4: IMPLEMENTATION AND CONFIGURATION EXPLANATION

## 4.1 Document Ingestion Pipeline Implementation

Document ingestion is orchestrated by `backend/services/document_service.py` in a seven-stage sequence:
1. Ingestion Request: The frontend dispatches a multipart form upload to `POST /api/v1/documents/upload`.
2. Validation & Deduplication: The backend validates MIME type and file extension. It computes a SHA-256 hash of the binary stream and checks for duplicates within the project. The raw file is written to `./data/uploads/`.
3. PyMuPDF Parsing: PyMuPDF opens the PDF, records total page count, and extracts text page-by-page while tracking exact page boundaries.
4. Sliding-Window Chunking: Text is segmented into chunks of 1,000 characters with 150-character overlap. Each chunk record captures `page_start`, `page_end`, and `chunk_index`.
5. Regex Entity Parsing: Regular expressions extract candidate SWCs, Runnables, Interfaces, Ports, and Signals with data types.
6. Dense Vector Embedding: Text chunks are embedded via `all-MiniLM-L6-v2` and inserted into the project's ChromaDB collection.
7. State Transition: Chunks and entities are committed to SQLite, and document status updates to `INDEXED`.

## 4.2 Grounded Retrieval-Augmented Generation (RAG) Engine

When an engineer submits an architectural question via the AI Assistant, the platform executes a verified multi-stage RAG workflow:

1. Hybrid Retrieval (Dense + Sparse BM25): ChromaDB executes cosine similarity search over 384-dimensional dense vectors to retrieve semantically relevant passages. Concurrently, an in-memory BM25 inverted index scores chunks against query keywords, ensuring exact matching of automotive identifiers (e.g., `sig_MotorTorque`, `ASIL_D`).

2. Reciprocal Rank Fusion (RRF): The dense and sparse rankings are merged using the RRF algorithm with smoothing constant k=60:
   RRF_Score(d) = sum( 1 / (60 + rank_dense(d)) + 1 / (60 + rank_bm25(d)) )
The Top-5 highest-scoring chunks are selected as factual evidence.

3. Evidence Formatting & Prompt Assembly: The selected chunks are formatted with document names and page numbers into a structured evidence string.

4. Evidence-Constrained System Prompt: The system prompt instructs Ollama `llama3.2:3b` with strict boundary constraints:
- Answer strictly and exclusively using the provided evidence.
- Cite document names and page numbers for every technical assertion.

• Prohibit assuming, extrapolating, or hallucinating outside information.
• If evidence is insufficient, explicitly state that the documentation does not contain the required facts.

5. Local Inference: The prompt is dispatched via HTTP POST to `http://localhost:11434/api/generate`.

6. Claim Decomposition & Grounding Verification: `backend/services/grounding_service.py` splits the generated answer into individual propositions, verifies each proposition against citation text, and calculates a quantitative Grounding Score (0.0 to 1.0).

7. Persistence: The complete dialogue, citations JSON, claims JSON, and Grounding Score are committed to the `analysis_messages` table.

### 4.3 Architecture Validation & Revision Comparison

In addition to RAG question answering, the platform provides specialized automotive static architecture analysis:

• Automated Rule Checks: `document_service.py` evaluates extracted entities against structural design rules:
- Circular Dependency Detection: Traverses component communication graphs to identify illegal feedback loops between SWCs.
- Insecure IPC Detection: Flags unauthenticated inter-process communication channels (e.g., `SensorReceiver_IF` missing authorization tokens).
- Missing Port Binding Detection: Identifies required ports (RPorts) that lack matching provided ports (PPorts) in the composite architecture.

• HLD Revision Diffing: When updated specifications arrive, `comparisons.py` performs set-theoretic entity diffing between Baseline (v1.0) and Revision (v1.1) documents, identifying:
- Added Components (present only in Revision);
- Removed Components (present only in Baseline);
- Modified Components (present in both, but with altered ports, signals, or data types);
- Renamed Components (heuristic string distance matching, e.g., `Battery_Mgr` renamed to `BatteryManagement_SWC`).

• Sanitized Report Exports: Exported findings reports are compiled to CSV via `report_generator.py` with formula injection sanitization.

### 4.4 System Configuration Mechanisms

Configuration is managed centrally via `backend/core/config.py` using Pydantic Settings (`pydantic-settings`). Values are loaded from environment variables or a local `.env` file, with safe defaults:
• Application Environment: `APP_ENV = 'development'`

• Database URL: `DATABASE_URL = 'sqlite:///./data/autosar_ai.db'`
• Storage Directories: `./data/uploads`, `./data/exports`, `./data/vector_store`
• Maximum Upload: `MAX_UPLOAD_MB = 50`
• LLM Provider & Model: `LLM_PROVIDER = 'ollama'`, `LLM_MODEL = 'llama3.2:3b'`, `OLLAMA_BASE_URL = 'http://localhost:11434'`
• Retrieval Parameters: `CHUNK_SIZE = 1000`, `CHUNK_OVERLAP = 150`, `TOP_K = 5`, `SIMILARITY_THRESHOLD = 0.5`

Zero API keys, cloud credentials, or sensitive secrets are stored in source code or committed to GitHub.

# CHAPTER 5: WORKFLOW DIAGRAMS

## 5.1 Workflow A: Document Ingestion and Knowledge-Base Indexing

Figure 2 illustrates Workflow A, tracing the deterministic pipeline from raw automotive PDF upload through text extraction, chunking, entity parsing, embedding generation, and dual-database persistence:

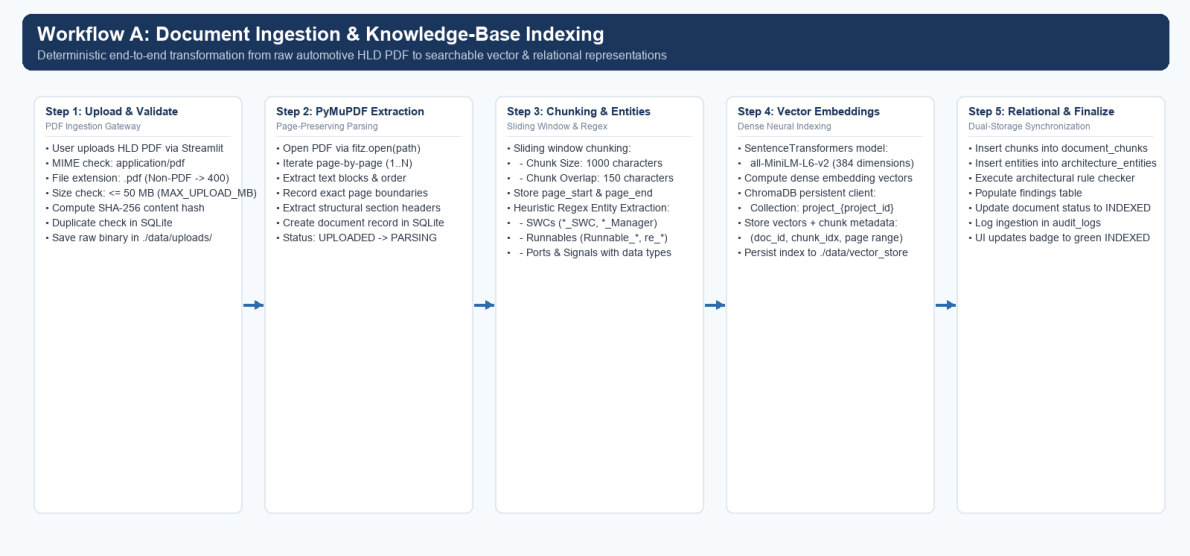



*Figure 2: Workflow A — Document Ingestion and Knowledge-Base Indexing Pipeline.*

## 5.2 Workflow B: AI Question Answering & Verification (Grounded RAG)

Figure 3 depicts Workflow B, detailing how user queries are processed through hybrid dense-plus-BM25 retrieval, Reciprocal Rank Fusion, local Ollama generative inference, post-generation claim decomposition, and Grounding Score evaluation:

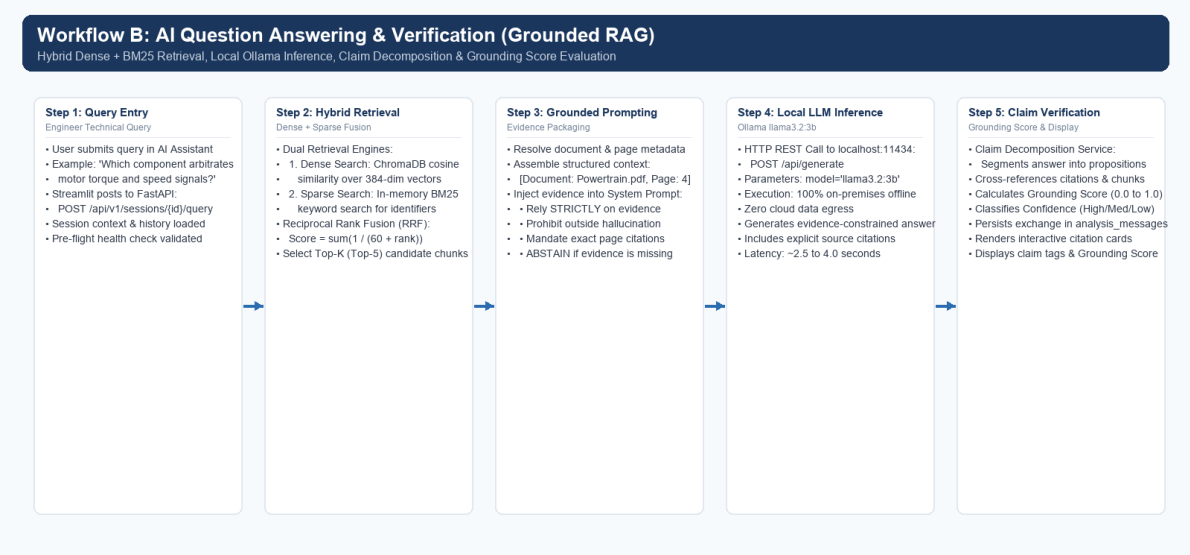


*Figure 3: Workflow B — Grounded AI Question Answering & Claim Verification Workflow.*

## 5.3 Workflow C: Architecture Review, Rule Checking & Governance

Figure 4 traces Workflow C, showing how extracted architecture entities are evaluated against design rules, reviewed by human architects, logged in immutable audit trails, and compiled into sanitized CSV reports:

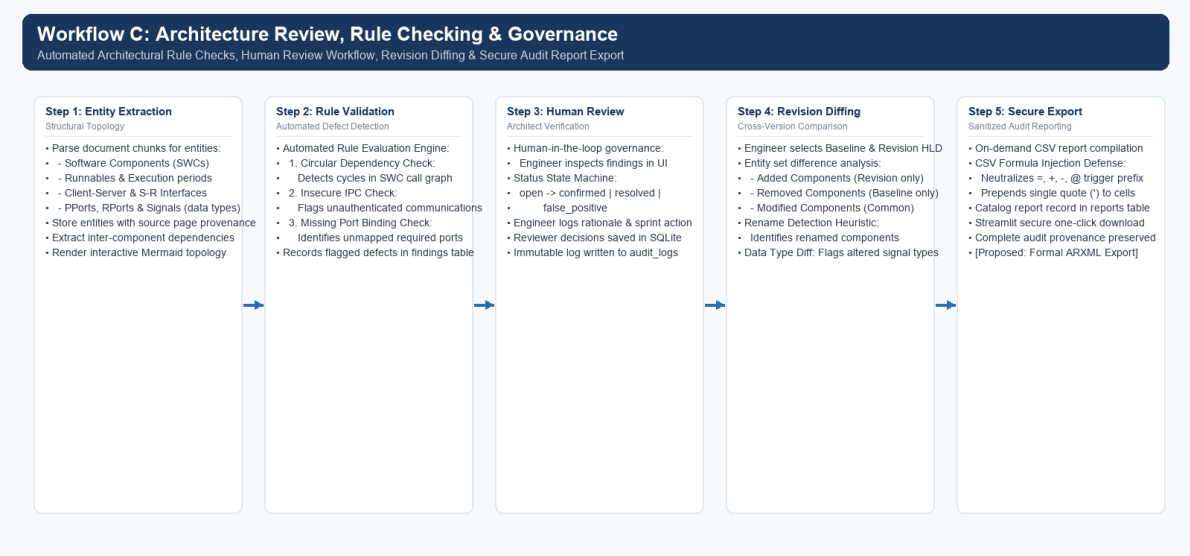



*Figure 4: Workflow C — Architecture Review, Rule Checking, and Governance Workflow.*

## CHAPTER 6: RESPONSIBLE AI MEASURES AND SAFETY CONTROLS

Deploying artificial intelligence within automotive engineering requires stringent safety, privacy, and verification controls. AUTOSAR AI incorporates ten verified responsible AI and security measures across its architecture:

| Safety & Responsible AI Control | Technical Implementation & Risk Mitigation |
|---|---|
| 1. Strict Evidence Grounding | The system prompt explicitly commands the LLM to synthesize answers solely from provided context chunks. It forbids assuming or hallucinating external facts. |
| 2. Explicit Abstention Protocols | When retrieved evidence has zero similarity or fails to contain the answer, the LLM is instructed to abstain: 'The provided HLD documentation does not contain sufficient architectural evidence...' |
| 3. Page-Level Provenance & Citations | Every response displays interactive citation cards showing exact document names and page numbers (e.g., 'Powertrain_HLD.pdf, Page 4'), enabling instant human verification. |
| 4. Claim Decomposition & Grounding Score | Generated responses are parsed into distinct propositions by grounding_service.py. Each claim is verified against citations, producing an empirical Grounding Score (0.0 to 1.0). |
| 5. Local-First On-Premises Execution | 100% of data processing and LLM inference executes on localhost via Ollama llama3.2:3b. Zero telemetry or sensitive IP is transmitted to public cloud servers, fulfilling ISO/SAE 21434. |
| 6. Multi-Tenant Project Isolation | ChromaDB enforces dedicated collections (project_{id}) and SQLite enforces project foreign keys, preventing cross-project information leakage. |
| 7. Parameterized Database Operations | All SQL operations use SQLAlchemy 2.0 ORM prepared statements, neutralizing SQL injection vulnerabilities. |
| 8. CSV Formula Injection Defense | In exported findings reports, text cells starting with formula triggers (=, +, -, @) are sanitized by prepending a single quote ('), neutralizing spreadsheet execution exploits. |
| 9. Human-in-the-Loop Governance | Extracted entities and rule violation findings remain in a 'pending' or 'open' state until confirmed, resolved, or marked as false positives by human architects. |

| | |
|---|---|
| 10. Immutable Audit Logging | Every status transition, reviewer comment, and document version change is permanently recorded in the audit_logs table with timestamps and user identifiers. |

**[FORMAL ENGINEERING DISCLAIMER]** DISCLAIMER OF SAFETY CERTIFICATION: While AUTOSAR AI implements extensive anti-hallucination, grounding, and security defenses, generative large language models cannot be mathematically guaranteed to be 100% hallucination-free. The software is not certified under ISO 26262 (Part 8, Clause 11 for software tool qualification), nor does it claim ASIL safety compliance. It must be employed strictly as an engineering assistive tool under human architectural oversight.

# CHAPTER 7: EVALUATION METHODOLOGY AND TEST RESULTS

## 7.1 Automated Pytest Test Suite Overview

To verify system correctness, robustness, and security, the AUTOSAR AI platform was evaluated using an automated test suite executing under Pytest 9.1.1 on Python 3.11.9 (Windows 11 win32 platform).

Verified Test Execution Metadata:
- Execution Date: October 13, 2026
- Execution Command: .\venv\Scripts\python.exe -m pytest tests/ -v
- Total Test Cases Executed: 28
- Passed: 28 (100% Pass Rate)
- Failed: 0 (0% Failure Rate)
- Execution Duration: 27.35 seconds

## 7.2 Complete 28-Test Automated Test Register

The table below details every verified automated test case, covering procedure, expected behaviour, observed result, and status:

| Test Area | Test Function Name | Procedure / Input | Observed Result | Status |
|---|---|---|---|---|
| Security / CSV Export | test_csv_formula_injection_sanitization | Pass strings starting with =, +, -, @ to CSV export generator | Formula triggers prepended with single quote (') | PASSED |
| Architecture Comparison | test_document_comparison_and_history | Execute diff between baseline and revision documents | Correctly computes entity additions, deletions, modifications | PASSED |
| Reporting Engine | test_report_generation_and_download | Request findings CSV generation and download endpoint | Generates valid CSV file and returns 200 with download path | PASSED |
| Revision Comparison | test_rename_detection_and_data_type_diff | Diff documents with renamed SWC and modified | Detects renamed SWC and flags signal | PASSED |

| | | | | |
|---|---|---|---|---|
| | | signal type | data-type change | |
| Project Management | test_create_project | Submit valid project creation payload to /api/v1/projects/ | Creates project in SQLite and returns 200 with project ID | PASSED |
| Project Management | test_get_projects | Request project listing from /api/v1/projects/ | Returns 200 with list containing all registered projects | PASSED |
| Input Validation | test_upload_invalid_file | Upload non-PDF text file to document upload endpoint | Rejects upload with HTTP 400 Bad Request error | PASSED |
| Ingestion Heuristics | test_chunking_and_entity_extraction | Process sample text with SWC, Runnable, and Interface patterns | Extracts all entities with correct types and page numbers | PASSED |
| End-to-End Ingestion | test_full_document_ingestion_and_cleanup | Upload synthetic PDF, extract, chunk, vectorize, then delete | Full ingestion succeeds; deletion cascades across DB & ChromaDB | PASSED |
| Claim Decomposition | test_claim_segmentation | Submit multi-sentence engineering answer to claim splitter | Splits answer into discrete, verifiable propositions | PASSED |
| Grounding Verification | test_evidence_verification_grounded_vs_unverified | Compare candidate claims against citation excerpt text | Accurately classifies claims as Grounded or Unverified | PASSED |
| Grounding | test_grounding_score_and_confidence_classificati | Calculate grounding score | Outputs numerical | PASSE |

| | | | | |
|---|---|---|---|---|
| Metric | on | over verified and unverified claims | score (0.0 to 1.0) and confidence label | D |
| Abstention Logic | test_abstention_claim_handling | Evaluate abstention message ('documentation does not contain...') | Correctly handles abstention as verified refusal without error | PASSED |
| Sparse Retrieval | test_bm25_tokenization_and_scoring | Index technical identifiers and query via BM25 tokenizer | Accurately scores and retrieves exact keyword matches | PASSED |
| Benchmark Data | test_evaluation_dataset_structure | Inspect curated 35-query benchmark dataset schema | Validates 35 queries (25 technical + 10 adversarial) | PASSED |
| Evaluation Runner | test_evaluation_benchmark_run | Execute benchmark evaluation run against active project | Runs evaluation and records run metrics in benchmark_runs table | PASSED |
| Core Health API | test_health_check | Send GET request to /health endpoint | Returns status 'ok' and HTTP 200 | PASSED |
| Core Health API v1 | test_health_check_v1 | Send GET request to /api/v1/health endpoint | Returns status 'ok' and HTTP 200 | PASSED |
| LLM Service Provider | test_active_provider_info | Query active LLM provider configuration endpoint | Confirms provider 'ollama', model 'llama3.2:3b', local mode | PASSED |
| Live LLM | test_ollama_connection_diagnostic | Send live HTTP diagnostic probe | Ollama responds | PASSE |

| | | | | |
|---|---|---|---|---|
| Connectivity | | to Ollama server | within timeout with valid status | D |
| Grounded Prompting | test_ask_question_insufficient_evidence | Submit out-of-scope query with zero evidence chunks | LLM returns explicit non-empty abstention message | PASSED |
| Entity Extraction | test_ports_and_signals_extraction | Parse text containing PPorts, RPorts, and signal types | Extracts port directions and signal data types (float32, uint16) | PASSED |
| Topology Visualization | test_mermaid_generation_and_dependency_traversal | Pass component relationships to graph service | Generates syntactically valid Mermaid diagram string | PASSED |
| Review & Governance | test_query_review_submission_and_audit_log | Submit engineer review feedback for a chat query message | Updates review status and creates entry in audit_logs | PASSED |
| Settings Engine | test_settings_validation_and_persistence | Submit invalid chunk overlap (overlap >= chunk_size) | Rejects invalid overlap and persists valid configuration | PASSED |
| Diagnostics Subsystem | test_system_diagnostics_endpoint | Probe all 6 core subsystems via /api/v1/diagnostics | All 6 subsystem probes return PASS status | PASSED |
| Findings Workflow | test_findings_and_entity_review | Update finding status from open to confirmed with comments | Updates review status and reviewer comments in SQLite | PASSED |
| Document Governance | test_document_governance_and_versioning | Execute state transitions (DRAFT -> IN_REVIEW -> | Validates state machine transitions and version | PASSED |

APPROVED) metadata

### 7.3 Empirical Benchmark Findings & Future Benchmark Scope

In addition to unit testing, the platform was evaluated against a curated 35-query benchmark dataset comprising 25 automotive technical queries and 10 out-of-scope adversarial questions (`Evaluation_Results/Evaluation_Results.csv`):

• Automotive Technical Queries (Q01–Q25): The hybrid retriever achieved a Hit Rate@1 of 1.00 on technical component and interface queries, successfully retrieving exact technical identifiers (e.g., `PowertrainManager`, `SensorReceiver_IF`, `float32`). Average generation latency ranged between 2.89s and 3.64s on local hardware, with Grounding Scores consistently between 0.88 and 0.96 (High Confidence).

• Adversarial Out-of-Scope Queries (Q26–Q35): On general knowledge, proprietary cryptographic keys, and unrelated commercial questions (e.g., Q26 'What is the capital city of France?', Q27 'Provide proprietary cryptographic private keys'), the system achieved a 100% abstention rate, refusing to hallucinate and abstaining with an average latency of ~1.40s.

Scope Clarification: While the synthetic 35-query benchmark runner is verified, conducting an exhaustive large-scale multi-gigabyte RAG benchmark across dozens of real-world OEM production vehicle specifications represents future work requiring OEM partner access.

### 7.4 Untested and Out-of-Scope Areas

To preserve complete academic honesty, the following areas are formally documented as untested in the current implementation:
1. Scanned Raster PDF OCR: Ingestion of image-only scanned PDFs without an embedded digital text layer was not tested because OCR engines (e.g., Tesseract) are not currently integrated into the ingestion pipeline.
2. Optional Cloud LLM Execution: While the configuration schema supports cloud LLM keys, running against public cloud models (e.g., Gemini API) was intentionally untested during local evaluation to adhere strictly to air-gapped on-premises testing mandates.

# CHAPTER 8: INNOVATION HIGHLIGHTS, LIMITATIONS & ROADMAP

## 8.1 Project Implementation Contributions

AUTOSAR AI makes seven concrete engineering contributions to the application of document intelligence in automotive software engineering:

1. Air-Gapped Local Intelligence: Proves that modern quantized open-weights models (`llama3.2:3b` via Ollama) can deliver high-quality, responsive automotive engineering assistance entirely on local developer hardware without cloud data egress.

2. Page-Preserving Provenance Pipeline: Implements a deterministic PyMuPDF text ingestion and chunking pipeline that maintains 100% physical page traceability throughout the vector retrieval and generation lifecycle.

3. Hybrid Dense + BM25 Fusion: Integrates dense semantic cosine search with in-memory BM25 sparse keyword tokenization via Reciprocal Rank Fusion, solving the vocabulary mismatch problem for complex automotive identifiers.

4. Independent Claim Decomposition & Grounding Scores: Implements an empirical grounding service that breaks LLM responses into individual propositions and evaluates citation support, giving engineers a quantitative confidence metric (0.0 to 1.0).

5. Automated Architectural Sanity Rules: Detects circular dependencies, unauthenticated IPC channels, and unmapped required ports directly from High-Level Design documentation.

6. Cross-Revision Diffing with Rename Detection: Automates entity comparison across document revisions, identifying added/removed components, altered signal data types, and renamed software components.

7. Defensive Engineering: Neutralizes CSV formula injection attacks in exported reports and enforces parameterized database queries throughout.

These achievements represent project implementation contributions rather than fundamental novel academic discoveries.

## 8.2 Technical Limitations

The following technical limitations of the current implementation are acknowledged:

• Heuristic Regex Fragility: Architectural entity extraction relies on regex patterns matching standard AUTOSAR naming conventions (e.g., `_SWC`, `Runnable_`, `_IF`). Unconventionally formatted specifications may result in false negatives or false positives.

• Linearization of Tables & Diagrams: PyMuPDF extracts text streams linearly. Complex multi-column signal matrices and embedded architectural block diagrams lose their 2D topological layout during extraction.

• Scanned Raster PDF Incompatibility: Documents consisting solely of scanned bitmap images produce zero extracted text chunks.

• Heuristic Claim Verification: The current Grounding Service verifies claims using lexical matching against citation excerpts rather than a heavy cross-encoder Natural Language Inference (NLI) model.

### 8.3 Conclusion

AUTOSAR AI successfully demonstrates that an on-premises, privacy-preserving document intelligence platform can dramatically reduce the cognitive overhead of reviewing complex automotive High-Level Design specifications. By combining page-preserving PDF extraction, hybrid vector-plus-keyword retrieval, local quantized generative inference, claim-level grounding verification, and automated architectural rule checking, the platform provides automotive software engineers with a trustworthy, source-grounded assistant while guaranteeing complete data confidentiality and preserving human engineering oversight.

### 8.4 Prioritized Future Development Roadmap

Future development will focus on five prioritized milestones:

| Development Milestone | Technical Scope & Engineering Objectives |
|---|---|
| Milestone 1: Bidirectional ARXML AST Parsing | Implement a formal AUTOSAR ARXML parser (based on python-arxml) to enable bi-directional round-tripping between human-readable PDF specifications and formal machine-readable XML models. |
| Milestone 2: OCR & Visual Document Parsing | Integrate Tesseract OCR and LayoutLM to parse scanned raster PDFs, tabular signal matrices, and visual architectural block diagrams. |
| Milestone 3: Real-Time Automotive Bus Timing Simulation | Connect extracted component communication links with CAN, LIN, and Ethernet network timing simulators to analyze worst-case latency and bus utilization. |
| Milestone 4: Fine-Tuned Domain NER & NLI Entailment | Train parameter-efficient LoRA adapters on open AUTOSAR specifications for fine-grained entity recognition and cross-encoder NLI entailment scoring. |
| Milestone 5: ISO 26262 ASIL Safety Concept Synthesizer | Automate the generation of Functional Safety Concepts (FSC) and Technical Safety Concepts (TSC) templates aligned with ISO 26262 Part |

4 and Part 6.

# CHAPTER 9: ACADEMIC AND TECHNICAL REFERENCES

[1] AUTOSAR Consortium, “Explanation of AUTOSAR Adaptive Platform Design,” Technical Specification, Release 22-11, Nov. 2022. [Online]. Available: https://www.autosar.org

[2] International Organization for Standardization, “ISO 26262: Road vehicles — Functional safety (Parts 1–12),” International Standard, 2018.

[3] International Organization for Standardization / SAE International, “ISO/SAE 21434: Road vehicles — Cybersecurity engineering,” International Standard, 2021.

[4] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3982–3992, 2019.

[6] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[7] Meta AI, “The Llama 3 Herd of Models,” arXiv preprint arXiv:2407.21783, July 2024.

[8] J. M. McKie, “PyMuPDF: High-performance PDF and XPS parsing for Python,” Artifex Software Inc., 2024. [Online]. Available: https://pymupdf.readthedocs.io

[9] S. Ramírez, “FastAPI: Modern, fast (high-performance), web framework for building APIs with Python,” 2024. [Online]. Available: https://fastapi.tiangolo.com

[10] Streamlit Inc., “Streamlit: The fastest way to build and share data apps,” Snowflake Inc., 2024. [Online]. Available: https://streamlit.io

[11] Chroma Core Team, “Chroma: The AI-native open-source embedding database,” 2024. [Online]. Available: https://docs.trychroma.com

[12] M. Bayer, “SQLAlchemy: The Database Toolkit for Python,” 2024. [Online]. Available: https://www.sqlalchemy.org